Queuing Lab

CS 5/4113

A. Workload Generator

Job Id	arrival time	Tasks
j0	0	["0: W 10", "1: W 10"]
j1	10	["0: W 10"]
j2	20	["0: W 10", "1: W 10", "2: W 10"]

Tasks are a list of **task** that are of the form:

- "<tid>: W N"

Where "N" is a positive number.

If Tasks == [] then the **Job** is done.

A. Workload Generator

Job Id	arrival time	Tasks
j0	20	[]
j1	30	[]
j2	60	[]

Note: When jobs are completed in our system we will use the same format, where arrival time will be the time the job left the system.

A. Workload Generator

Workload Generator:
Input: interval, distribution (uniform, normal, geometric, Poisson)
Output: Synthetic workload of sorted jobs



Workload

Job Id	arrival time	Tasks
j0	0	["W 10", "W 10"]
j1	10	["W 10"]
j2	20	["W 10", "W 10", "W 10"]

- An interval will define the starting and ending range of time steps.
- The number of jobs arriving into the system should be drawn from the distribution
- All inputs to workload generator should be command line arguments (including option arguments).
- Individual tasks for the workload should have an option to be fixed or drawn from the same distribution.

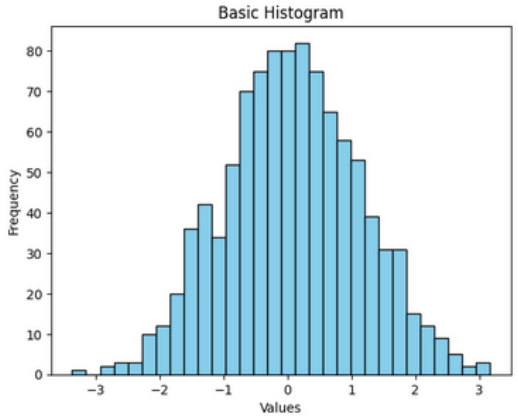
A. Implement a Workload Visualizer (py/r)

Workload

Job Id	arrival time	Tasks
j0	0	["W 10", "W 10"]
j1	10	["W 10"]
j2	20	["W 10", "W 10", "W 10"]

Workload Visualizer

--hist="arrival time" --out="f.png"

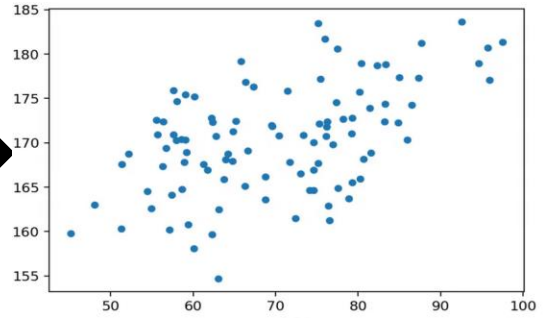


Workload

Job Id	arrival time	Tasks
j0	0	["W 10", "W 10"]
j1	10	["W 10"]
j2	20	["W 10", "W 10", "W 10"]

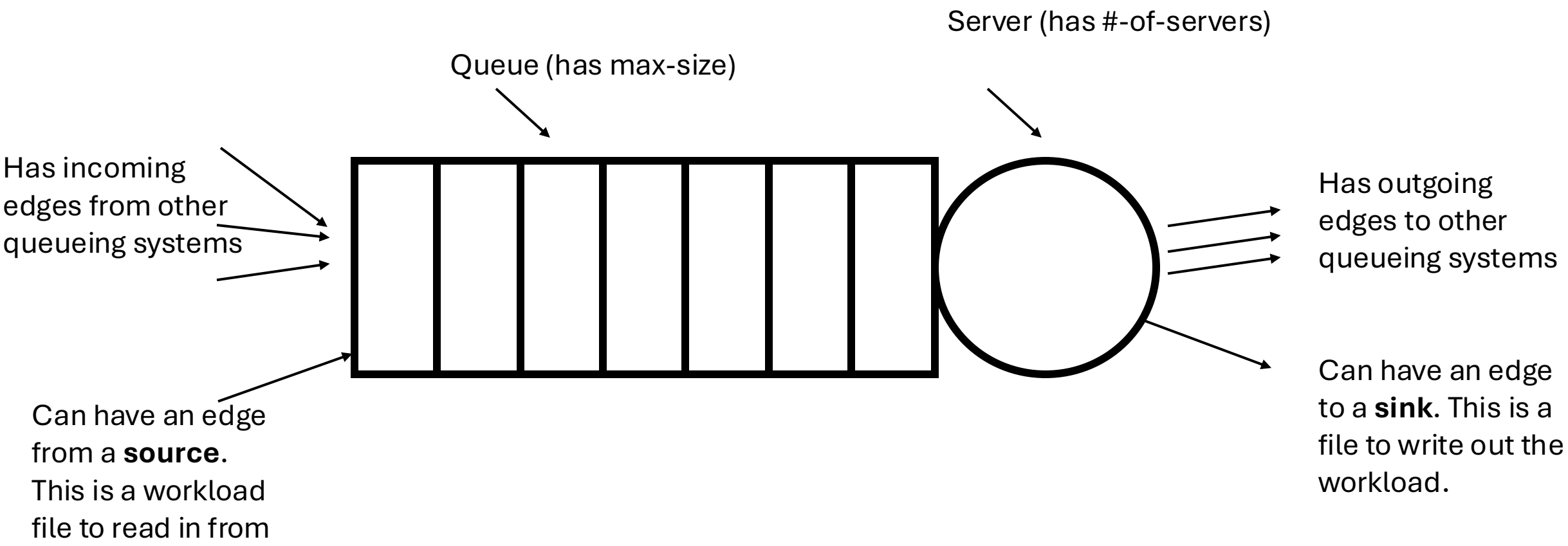
Workload Visualizer

--scat="arrival time, len(Tasks)" --out="f.png"



- The workload visualizer should take as an input the filename of a workload, along with optional plotting parameters (x/y-limits, titles)
- In histogram mode it should take the name of the column to visualize (arrival time) along with "bin count" and produce a histogram
- In scatter plot mode it should take the name of two columns (arrival time and "len(Tasks)") and produce a scatter plot.
- Matplotlib and pandas can do all the lifting here.

B. Simple Queueing System



B. Simple Queueing System: Queue Configuration

Queue System Configuration

Queue sys ID	Max Queue Size	# of servers	Server Function	Server Function Rate	source	sink	Edges in	Edges out
q0	8	1	Sub-10	10	in0.wl	out0.wl	[]	[]

B. Simple Queueing System: Server Functions

Server Function takes task list <**Tasks**>
and returns a modified task list <**Tasks**>
after <**Server Function Rate**> time steps.

Example: **Sub-X**
Task-out = **Sub-X** (Task-in)
Subtracts X from the first task in Task-In or
removes first task if N <= X in "W N"

Task-Out = Sub-10(Task-In)

Task-In	Task-Out
["0: W 10"]	[]
["0: W 20"]	["0: W 10"]
["0: W 10", "1: W 10"]	["1: W 10"]
[]	[]

B. Simple Queueing System: Server Functions

Server Function can be chained together so multiple functions are applied over multiple <**Server Function Rate**> time steps.

Example: Sub-5,Sub-10
Apply Sub-5 over <**Server Function Rate**> time steps, then apply Sub-10 over another <**Server Function Rate**> time steps.

Task-Out = Sub-10(Sub-10(Task-In))

Task-In	Task-Out
["0: W 10"]	[]
["0: W 20"]	["0: W 10"]
["0: W 10", "1: W 10"]	["1: W 10"]
[]	[]

B. Simple Queueing System: Queue Metrics

Queue Metrics (qmet.csv)

time-step	queue-sys-id	num-jobs-in-queue	num-jobs-in-service	Num-of-errors
0	0	1	1	0

At every time step each queueing system will produce a line of values at that step:

- **time-step – current time step**
- **queue-sys-id – ID of the queueing system reporting**
- **num-jobs-in-queue - (L or Saturation) how many jobs are waiting for a server?**
- **num-jobs-in-service – how many jobs are in a server?**
- **num-of-errors – number of jobs dropped (no room in queue, no outgoing edge)**

This file will have a lot of rows (num-time-steps*num-of-queues)

B. Simple Queueing System: Queue Metrics

Queue Metrics (qmet.csv)

time-step	queue-sys-id	num-jobs-in-queue	num-jobs-in-service	Num-of-errors
0	0	1	1	0
1	0	1	1	0
2	0	1	1	0
3	0	2	1	0
4	0	3	1	0
5	0	3	1	1

B. Simple Queueing System: Event Logs

Any time a job in the system changes state it will be logged. We will use this information to characterize problematic jobs.

The format for each line of the log file is as follows:

- **"<time> <job-id> IN <qsid> ARRIVE-VIA <edge>" -** job enters a queueing system via edge (internal/source)
- **"<time> <job-id> IN <qsid> DEPART-VIA <edge>"** - job leaves a queueing system via edge (internal/sink)
- **"<time> <job-id> IN <qsid> ENTERS-SERVER <sid>" -** job enters server <sid>
- **"<time> <job-id> IN <qsid> EXITS-SERVER <sid>" -** job exits server <sid>
- **"<time> <job-id> IN <qsid> TASK-START <tid> <task>" -** task at <tid> starts
- **"<time> <job-id> IN <qsid> TASK-STOP <tid> <task>" -** task at <tid> stops
- **"<time> <job-id> IN <qsid> ERROR <err-type>" -** <err-type> is QUEUE-FULL,NO-OUT-EDGE

Collecting this information on all jobs will allow us to understand the system from the perspective of the jobs. For example, if we want to understand errors we can grep the log for ERROR then grep the <job-id> for those respective jobs.

B. Simple Queueing System: Event Logs

Worklist in0.wl

Job Id	arrival time	Tasks
j0	0	["0 W 10", "1 W 10"]
j1	10	["0 W 10"]
j2	20	["0 W 10", "1 W 10", "2 W 10"]

Queue Assumptions: All tasks are complete before job leaves system. (This will not always be the case)

The server function is:
Sub-10*
Where Sub-10 is performed 0 or more times

Sample Event Log

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"

B. Simple Queueing System

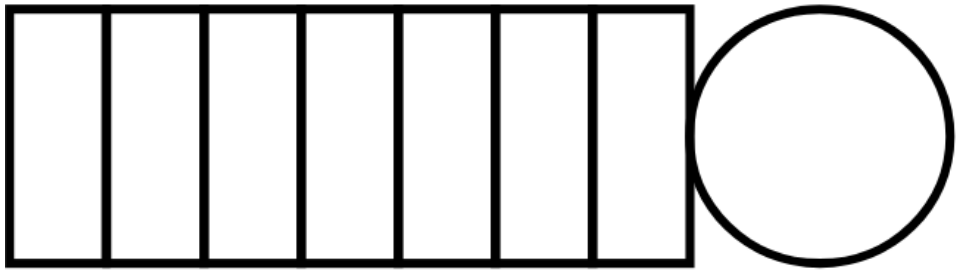
Source Workload (in0.wl)

Job Id	arrival time	Tasks
j0	0	["W 10", "W 10"]
j1	10	["W 10"]
j2	20	["W 10", "W 10", "W 10"]

Queue Configuration

ID	Max Queue Size	# of servers	Function	sinks	sources	Edges in	Edges out
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]

Clock: 0



Sink Workload (out0.wl)

Job Id	arrival time	Tasks
j0	20	["Done"]
j1	30	["Done"]
j2	60	["Done"]

Queue metrics (qmet.csv)

time-step	queue-sys-id	num-jobs-in-queue	num-jobs-in-service	Num-of-errors
0	0	1	1	0
1	0	1	1	0
2	0	1	1	0
3	0	2	1	0
4	0	3	1	0
5	0	3	1	1

Job Events (job.log)

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"

Sketch of Queue Simulator algorithm

Set t = 0

While time step t < time limit

- For Each Queue System Qs
  - Move completed jobs out
    - (if job is "Done" and sink exists, move to sink)
    - Otherwise, assign to outgoing edge according to edge weight (equal if unspecified)
  - Update time remaining on jobs in service (server.time_left--)
  - Send new jobs from queue to empty servers
  - Add new jobs with arrival 't'
    - From sources
    - From incoming edges
- t=t+1

C. Tooling: Filtering Queue Metrics

Queue metrics (qmetin.csv)

time-step	queue-sys-id	num-jobs-in-queue	num-jobs-in-service	Num-of-errors
0	0	1	1	0
0	1	1	1	0
1	0	1	1	0
1	1	2	1	0
2	0	3	1	0
2	1	3	1	1

./filter-by-queue
qmetin.csv
qmetout.csv
--queues=0

Queue metrics (qmetout.csv)

time-step	queue-sys-id	num-jobs-in-queue	num-jobs-in-service	Num-of-errors
0	0	1	1	0
1	0	1	1	0
2	0	3	1	0

Remove all but the queues specified by: "--queue=a,b,c,..."

C. Tooling: Filtering Event Logs

Event Log (jobin.log)

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"



```
./filter-by-job
jobin.log
jobout.log
--jobs=0
```

Remove all but the queues specified by: "--jobs=a,b,c,..."



Event Log (jobout.log)

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0

C. Queue Utilization Saturation Errors: Compute USE

Queue metrics (qmet.csv)

time-step	queue-sys-id	num-jobs-in-queue	num-jobs-in-service	Num-of-errors
0	0	1	1	0
0	1	1	1	0
1	0	1	1	0
1	1	2	1	0
2	0	3	1	0
2	1	3	1	1

./compute-use
qmet.csv
quse.csv

Queue use metrics (quse.csv)

time-step	Utilization(10)	Saturation(10)	Errors(10)	Utilization(100)	Saturation(100)	Errors(100)	Utilization(1000)	Saturation(1000)	Errors(1000)
0	0	1	1	0	1	1	0	1	1
1	0	1	1	0	1	1	0	1	1
2	0	1	1	0	1	1	0	1	1
3	0	2	1	0	2	1	0	2	1
4	0	3	1	0	3	1	0	3	1
5	0	3	1	0	3	1	0	3	1

For each Queueing System we want to compute Average Utilization, Average Saturation, and Total Errors Seen over different time scales (sliding window of 10, 100, 1000 steps).

- Utilization is the % of time a job is in server.
- Saturation is the number of jobs queueing
- Errors is the rate of errors originating in that queue.

See https://www.brendangregg.com/usemethod.html
https://blog.danslimmon.com/2019/03/25/latency-and-throughput-optimized-clusters-under-load/

time-step	Utilization(10)	Saturation(10)	Errors(10)	Utilization(100)	Saturation(100)	Errors(100)	Utilization(1000)	Saturation(1000)	Errors(1000)
0	0	1	1	0	1	1	0	1	1
1	0	1	1	0	1	1	0	1	1
2	0	1	1	0	1	1	0	1	1
3	0	2	1	0	2	1	0	2	1
4	0	3	1	0	3	1	0	3	1
5	0	3	1	0	3	1	0	3	1

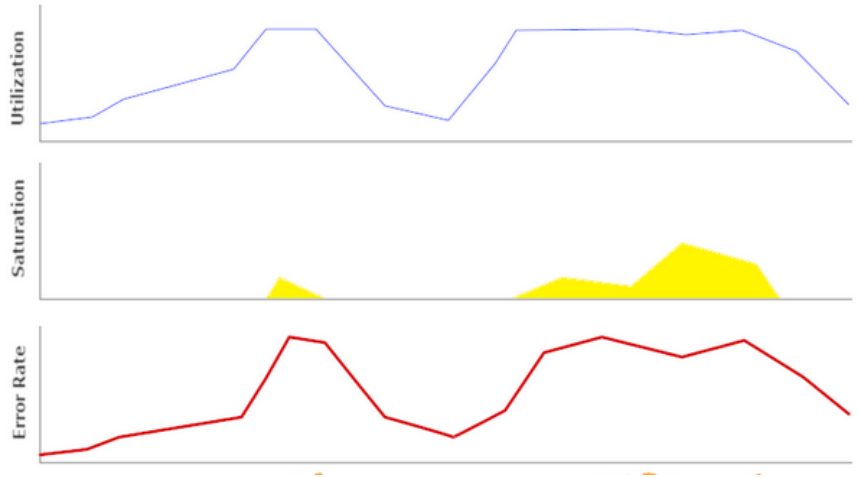
C. Queue Utilization Saturation Errors: Plot USE

Queue use metrics (quse.csv)

time-step	Utilization(10)	Saturation(10)	Errors(10)	Utilization(100)	Saturation(100)	Errors(100)	Utilization(1000)	Saturation(1000)	Errors(1000)
0	0	1	1	0	1	1	0	1	1
1	0	1	1	0	1	1	0	1	1
2	0	1	1	0	1	1	0	1	1
3	0	2	1	0	2	1	0	2	1
4	0	3	1	0	3	1	0	3	1
5	0	3	1	0	3	1	0	3	1

./plot-use
quse.csv
use.png

USE Plot (use.png)



See https://www.brendangregg.com/usemethod.html
https://blog.danslimmon.com/2019/03/25/latency-and-throughput-optimized-clusters-under-load/

Using the USE metrics create plots for:

- Utilization vs Time Step
- Saturation vs Time Step
- Error Rate vs Time Step

I recommend that each queue has its own color, and the the 10,100,1000 interval are shades of the same color. We will stack these plots along with Job response time (vs time-step) and throughput (vs time-step) for analysis.

C. Events: Derived System Metrics

Event Log (jobin.log)

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"

./derive-sys-metrics jobin.log sysmetric.csv

Derived System Metrics (sysmetric.csv)

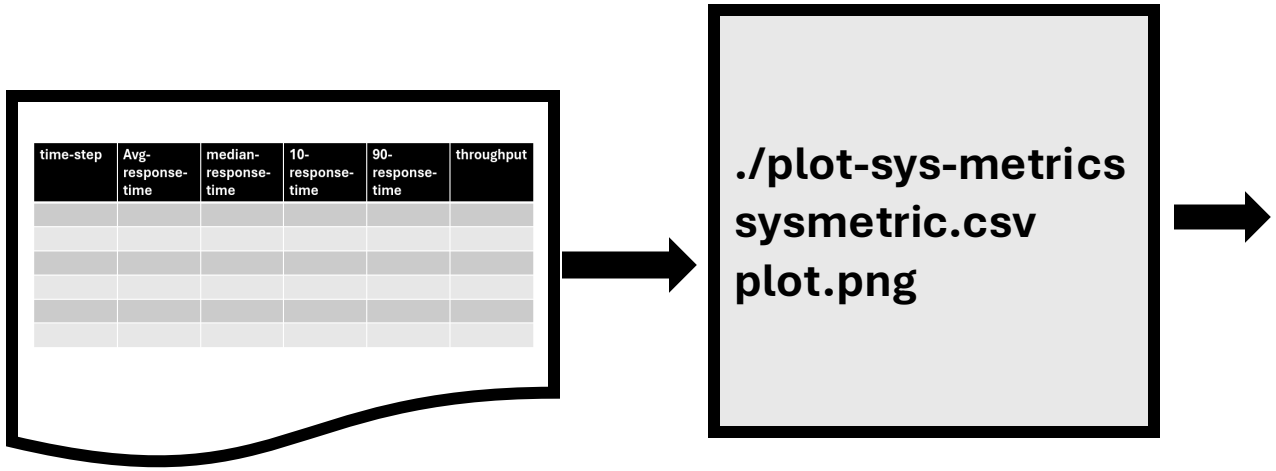
time-step	Avg-response-time	median-response-time	10-response-time	90-response-time	throughput

Using the event log, you will track statistics for jobs entering and leaving the simulator via sources and sinks. In particular you will build a time series including:

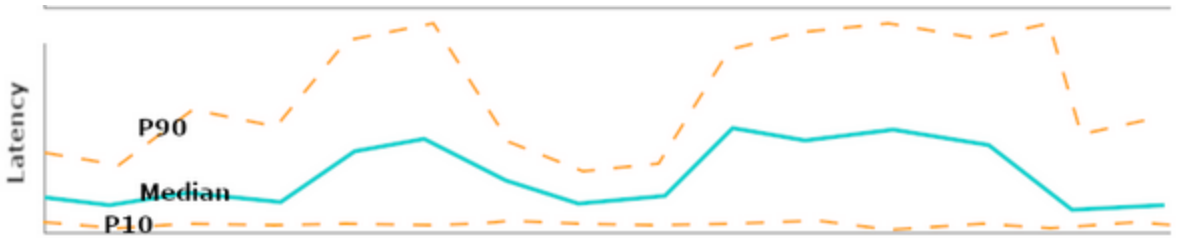
- time-step
- Average response-time of jobs exiting at time-step
- Median response-time of jobs exiting at time-step
- 90th percentile response-time of jobs exiting at time-step
- 10th percentile response-time of jobs exiting at time-step
- Throughput (# of jobs exiting) at time-step

C. Events: Plot System Metrics

Derived System Metrics (sysmetric.csv)



Response time (latency) vs time-step



Throughput vs time-step



We will reconcile these metrics against the USE metrics in order to draw conclusions about the behavior of our system. Typically when we monitor a system (or build a dashboard) these (USE, throughput, response time) are what we will track.

https://blog.danslimmon.com/2019/03/25/latency-and-throughput-optimized-clusters-under-load/

C. Events: Derived Job Metrics

Event Log (jobin.log)

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"

./derive-event-metrics jobin.log emetric.csv

Event Log (emetric.csv)

Job id	Queue-sys id	Arrival-time	Time-in-queue	Time-in-service	response-time	Turn-around-time
0	0					
0	1					
1	0					
2	1					
3	0					
3	0					

For every job that enters a queue we want to compute:

- Job id
- Queue system id
- arrival-time-in-queue
- time-in-queue
- time-in-service
- response-time (time until it first enters the server)
- turn-around time (time-in-queue+time-in-service)

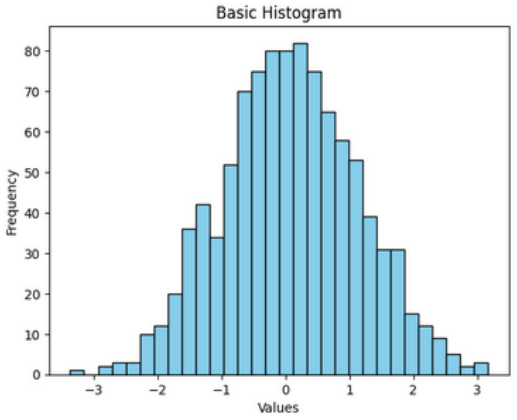
A job may enter a queue multiple times, and each visit would represent a row.

C. Events: Plotting Derived Job Metrics

Event Log (emetric.csv)

Job id	Queue-sys id	Arrival-time	Time-in-queue	Time-in-service	response-time	Turn-around-time
0	0					
0	1					
1	0					
2	1					
3	0					
3	0					

./plot-derive-event-metrics emetric.csv plot.png --hist="arrival-time"

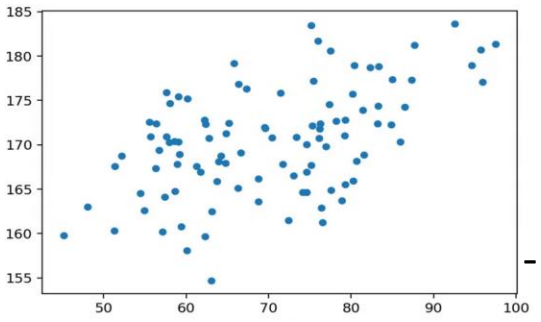


- The visualizer should take as an input the filename of a workload, along with optional plotting parameters (x/y-limits, titles)
- In histogram mode it should take the name of the column to visualize (arrival time) along with "bin count" and produce a histogram

Event Log (emetric.csv)

Job id	Queue-sys id	Arrival-time	Time-in-queue	Time-in-service	response-time	Turn-around-time
0	0					
0	1					
1	0					
2	1					
3	0					
3	0					

./plot-derive-event-metrics emetric.csv plot.png --scat="arrival-time,response-time"



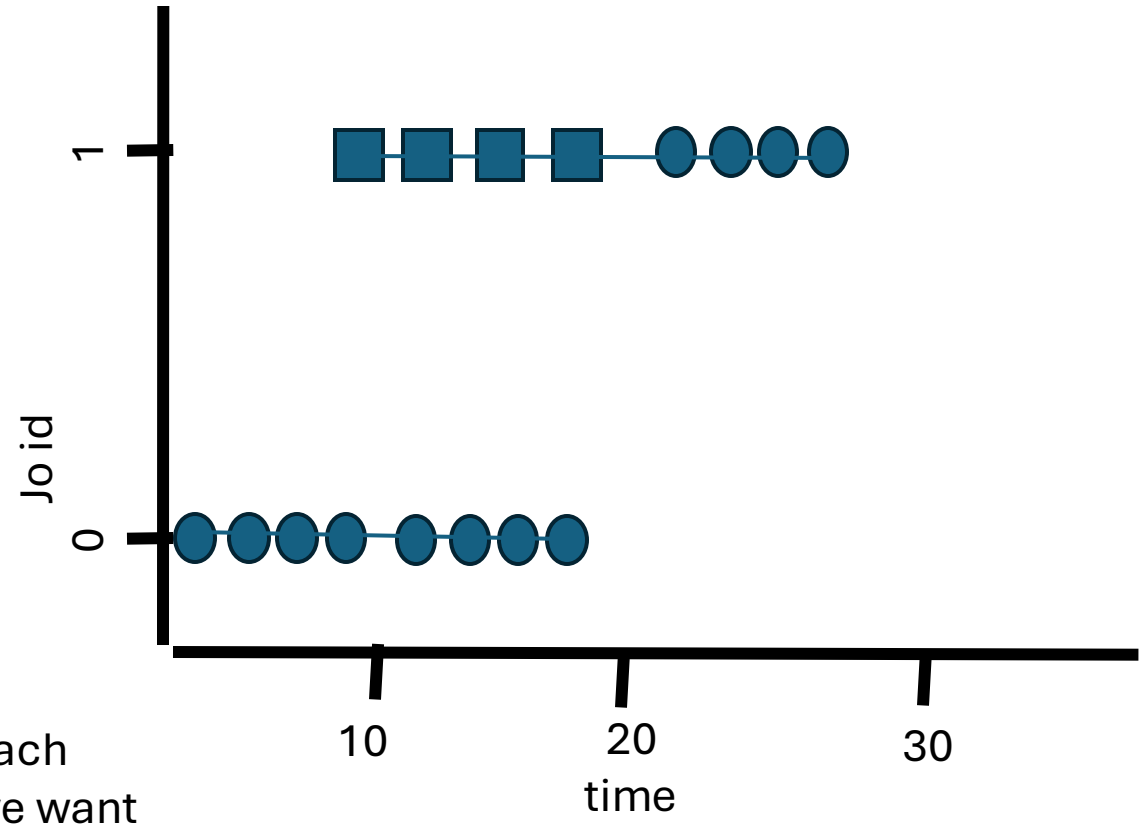
- In scatter plot mode it should take the name of two columns (arrival time and response time) and produce a scatter plot.
- Matplotlib and pandas can do all the lifting here.

C. Events: Tracing

Event Log (jobin.log)

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"

./create-event-trace
jobin.log
out.png



Each job is traced on a plot where each event is represented. In particular we want to know when a job is in a queue (box marker), in service (circle marker), or error (x marker). Additionally, the position of the line on the y-axis will represent the job id. Color can be used to represent the current queue.

D. Single Queue Modelling and Analysis

- Pick a real-world process to model as a single queue (ex: aircraft maintenance, Factorio).
- How do you map that process in terms of a queueing problem?
- What are the metrics that matter?
- How do map that queueing problem in terms of the simulator?
  - How do you configure the files?
  - How do you generate the workload and why?
  - How will you interpret the behavior (USE, throughput, response time/latency)?
- Run the simulation at various scales.
  - How does the system behave?
  - Does this make sense in the context of the original problem?
- Sensitivity Study:
  - How do you improve response-time/throughput with +25% resource, +50% resource, +100% resources? (resources can be servers, queue depth, links between queues).
  - How do you destroy response-time/throughput with –10%, -20% , -30% resources.
- What takeaways would you pass to someone building this real-world process?

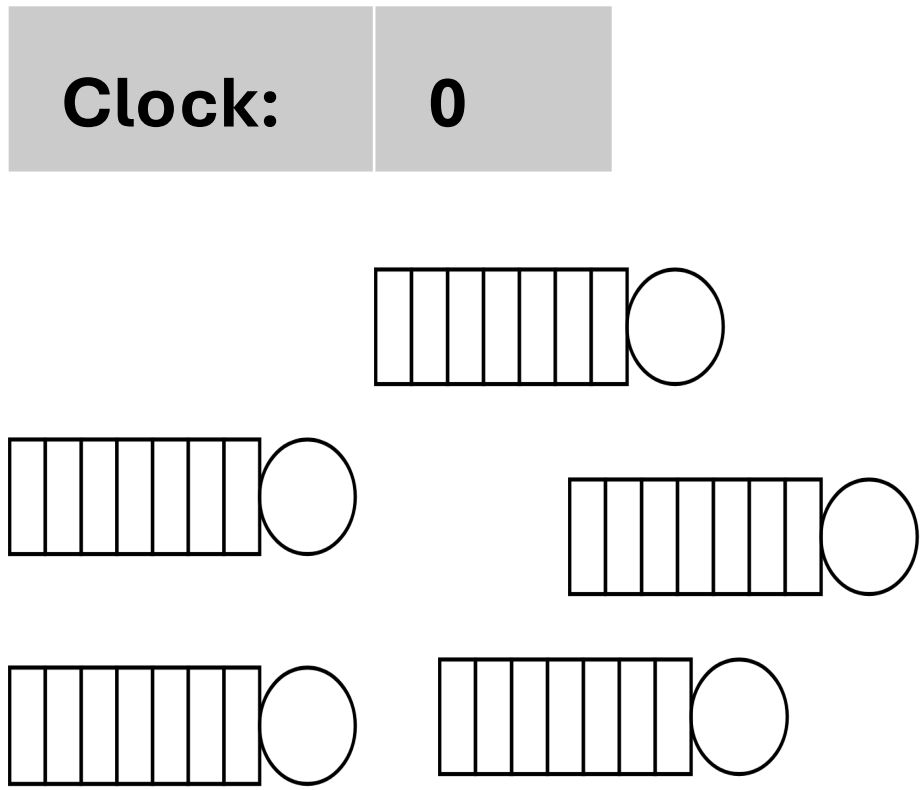
E. Multi Queueing System

Source Workloads (inX.wl)

Job Id	arrival time	Tasks
j0	0	["W 10", "W 10"]
j1	10	["W 10"]
j2	20	["W 10", "W 10", "W 10"]

Queue Configuration

ID	Max Queue Size	# of servers	Function	sinks	sources	Edges in	Edges out
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]



Sink Workloads (outX.wl)

Job Id	arrival time	Tasks
j0	20	["Done"]
j1	30	["Done"]
j2	60	["Done"]

Queue metrics (qmet.csv)

time-step	queue-sys-id	num-jobs-in-queue	num-jobs-in-service	Num-of-errors
0	0	1	1	0
1	0	1	1	0
2	0	1	1	0
3	0	2	1	0
4	0	3	1	0
5	0	3	1	1

Job Events (job.log)

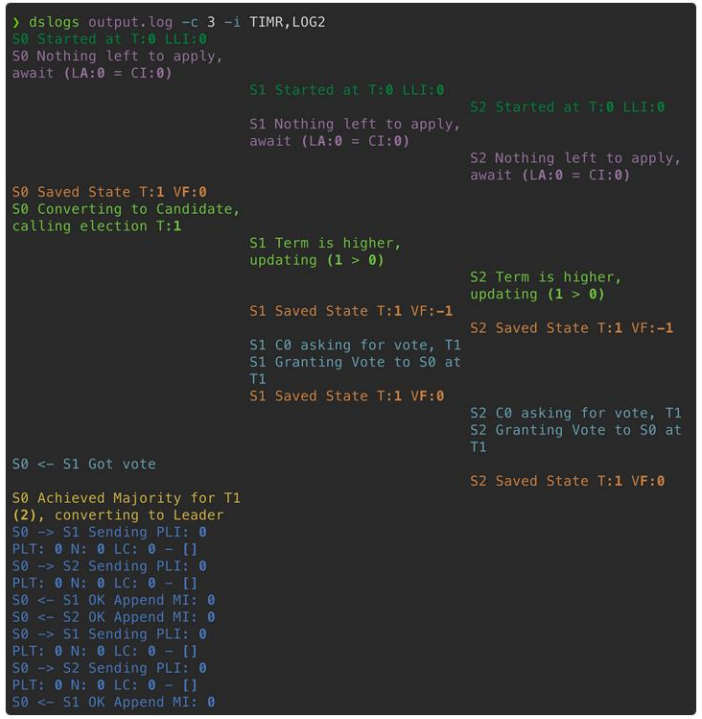
Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"

E. Event Log Pretty Printer

Event Log (jobin.log)

Event Log
0 j0 IN q0 ARRIVE-VIA in0.wl
0 j0 IN q0 ENTERS-SERVER 0
0 j0 IN q0 TASK-START 0 "W 10"
10 j0 IN q0 TASK-END 0 "W 10"
10 j0 IN q0 TASK-START 1 "W 10"
10 j1 IN q0 ARRIVE-VIA in0.wl
20 j0 IN q0 TASK-END 1 "W 10"
20 j0 IN q0 EXITS-SERVER 0
20 j1 IN q0 ENTERS-SERVER 0
20 j1 IN q0 TASK-START 0 "W 10"
20 j2 IN q0 ARRIVE-VIA in0.wl
30 j1 IN q0 TASK-END 1 "W 10"

./pretty-print-events jobin.log



Create a tool that arranges the output of the event log in an easy to interpret representation. This tool will layout the events with rows corresponding to time-step (multiple rows may be needed for a single time-step). Columns will represent different queueing systems, and color will be used for event type.

Follow the example from https://blog.josejg.com/debugging-pretty/

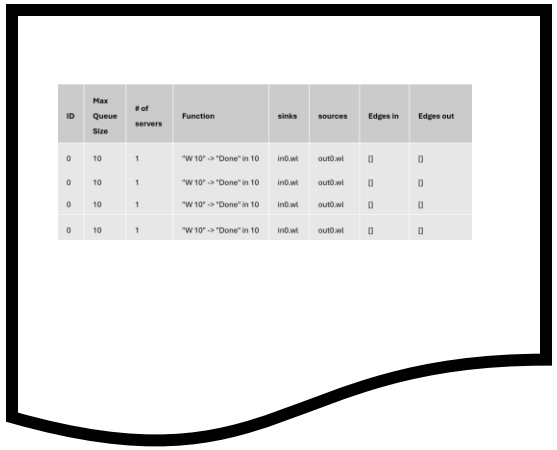
E. Network Config Generator

Network from https://snap.stanford.edu/data/index.html

```
# Undirected graph (each pair of nodes is saved twice):as19971108.txt
# Autonomous systems (from the BGP logs) - 11 08 1997
# Nodes:3015    Edges: 5347
# FromNodeId    ToNodeId
1       1740
1       5413
1       701
1       286
1       1239
1       3561
1       2914
1       2548
1       6478
1       71
```

./create-config-from-network network.mtx qconf.csv

Queue Configuration



ID	Max Queue Size	# of servers	Function	sinks	sources	Edges in	Edges out
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]
0	10	1	"W 10" -> "Done" in 10	in0.wl	out0.wl	[]	[]

This will allow you to create very large complex networks of queues from network files of real-world graphs. Most network files are in a sparse matrix file format called MTX which stores each edge (u v [w]) on a row. Libraries exists to read this that work to varying degrees, but this is not a hard format to parse if you implement your own parser.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.io.mmread.html

https://scanpy.readthedocs.io/en/stable/generated/scanpy.read_mtx.html

https://snap.stanford.edu/snappy/

F. Multiqueue Modelling and Analysis

- Pick a real-world process to model as a network of queues. These will be at multiple scales and you will have to generate the larger ones from network (graph) files of real systems (https://snap.stanford.edu/data/index.html).
  - 10s Queues 1000-10000 jobs (Example: a CPU Fetch/Decode/Execute/ (SMALL)
  - 100s Queues 10000-100000 jobs (MEDIUM)
  - 1000s Queues 100000-1000000 jobs (LARGE)
- How do you map that process in terms of a queueing problem?
- What are the metrics that matter?
- How do map that queueing problem in terms of the simulator?
  - How do you configure the files?
  - How do you generate the workload and why?
  - How will you interpret the behavior (USE, throughput, response time/latency)?
- Run the simulation at various scales.
  - How does the system behave?
  - Does this make sense in the context of the original problem?
- Sensitivity Study:
  - How do you improve response-time/throughput with +25% resource, +50% resource, +100% resources? (resources can be servers, queue depth, links between queues).
  - How do you destroy response-time/throughput with –10%, -20% , -30% resources.
- What takeaways would you pass to someone building this real-world process?

G. Technical Report

- Introduction (What are you trying to do and why is it important?)
- Theory
- Mechanism (How did you implement this theory?)
- Analysis and Results
  - Single Queue
  - Multi-Queue
- Summary

G. Presentation

- Analysis and Results of the problems you mapped to the simulator.
  - Single Queue
    - Mapping
    - Results
    - Sensitivity
  - Multi-Queue
    - 10s Mapping/results/sensitivity
    - 100s Mapping/results/sensitivity
    - 1000s Mapping/results/sensitivity